

Productive Failure Report

Deliverable 3 — Naive Baseline

Conversational Memory Intelligence System

Hypothesis

We believe a memory system that (1) saves everything with no filtering, (2) finds memories using only plain keyword matching, (3) never cleans up old or replaced facts, and (4) fits whatever it finds into the prompt with no smart prioritization, will fail in three specific, predictable ways:

- It will not be able to tell which of two conflicting facts about the user is currently true.
- It will save and casually repeat back sensitive information, since nothing ever filters what gets stored.
- When there isn't enough room to fit everything, it will drop information based on arbitrary ordering, not on what actually matters — sometimes keeping a useless fact and losing the correct one.

Baseline Design

The baseline is deliberately the simplest thing that could be called a working memory system — "naive" here means minimal, not artificially broken:

- **Storage:** every message is saved. Nothing is judged as important or unimportant, nothing is ever deleted automatically.
- **Retrieval:** a saved memory is returned if it shares at least one meaningful word with the question (a small list of common words like "the," "is," "was" is ignored so it isn't just noise-matching). No embeddings, no semantic understanding, no ranking by anything except how many words overlap.
- **Lifecycle:** no consolidation, no expiration, no correction.
- **Context injection:** retrieved memories are added to the prompt in overlap-ranked order until a character budget runs out, then it simply stops — no attempt to swap in a more useful, smaller memory instead.
- **Answer generation:** the retrieved memories and the question are sent to a real model — Claude (`anthropic/claude-sonnet-5`), via OpenRouter, temperature 0 — so we are testing the whole pipeline's real behavior, not just the retrieval step in isolation.

This matches the historical pattern identified in Deliverable 1 as "Approach 6: search for facts that sound related" — the most naive retrieval-based fix anyone would reach for first.

Workload

10 fixed test cases (never randomly generated), covering every situation the handbook requires: irrelevant memories, a contradiction with no time signal, two changed-preference cases, a long conversation with no budget limit, a long conversation with a tight budget, two multi-user isolation cases, one sensitive-information case, and one cold-start case. Full details and exact wording are in `baseline_protocol.md` and `naive_baseline/fixtures.py`.

Model: `anthropic/claude-sonnet-5` · temperature 0 · max output 300 tokens. Retrieval itself has no randomness — the same 10 cases produce the same retrieved memories every time.

Results

#	Situation	Outcome	Notes
1	Irrelevant memories	Pass	Weather/movie noise correctly ignored; answered Python correctly.
2	Contradiction, no time signal	Fragile pass	Claude noticed the conflict and asked for clarification — good outcome, but the system itself has no mechanism for this; it relied entirely on the model's own judgment.
3 (original wording)	Changed preference (food)	Inconclusive	The word "now" in the memory text gave away the answer. Claude answered correctly, but this proved Claude can read, not that the system understands recency.
3 (corrected wording)	Changed preference (food), no time-clue	Fail	With the giveaway word removed, Claude correctly stated it had no way to tell which fact was current. The system cannot answer a basic personal-preference question when nothing marks which fact is newer.
4	Changed preference (project language)	Inconclusive	Same issue as case 3 — "switched to" gave away the answer in plain English.
5	Long conversation, no budget limit	Pass	An important fact stated early survived 20 unrelated messages, because keyword search has no concept of "old" — it checks every saved memory regardless of position.
6	Long conversation, tight budget	Fail	A vague, irrelevant decoy memory (which also happened to contain the word "deadline") was kept, while the real deadline fact was silently dropped by the budget cutoff. The

			final answer could not state the actual deadline, even though it had been told to the system.
7	Multi-user isolation (color)	Pass	No cross-user leakage.
8	Multi-user isolation (role)	Pass	No cross-user leakage.
9	Sensitive info retained	Fail	A password shared in passing was saved with no filtering, and handed straight back when asked, with no admission-time safeguard.
10	Cold start	Pass	Correctly reported no saved information, instead of inventing an answer.

Latency across the original 10-case run: p50 $\approx$ 3.6s, p95 $\approx$ 4.8s per question (see `baseline_results.csv` for exact figures). Storage after all 10 cases: 56 saved memories, 2,475 characters total.

Failure Taxonomy

Failure 1 — no sense of time.

Observed: given two contradicting facts with no time-clue, the system could not determine which was current.

Violated assumption: that keyword matching alone is enough to answer everyday questions about the user's current state.

Impact: the system cannot reliably answer basic personal questions once a preference changes — undermining its core purpose.

Failure 2 — no admission-time filtering.

Observed: a password was saved with no judgment about sensitivity, and repeated back on request.

Violated assumption: that anything the user says is safe to store and retrieve verbatim.

Impact: a real, concrete privacy/security risk — the kind explicitly called out as a constraint in `problem_reconstruction.pdf`.

Failure 3 — blind budget cutoff.

Observed: under a tight memory budget, a useless decoy fact was kept while the correct, relevant fact was silently dropped.

Violated assumption: that "whatever fits, in whatever order it was found" is an acceptable way

to decide what stays in context.

Impact: the system can lose information the user already gave it, producing a worse answer than if the memory feature didn't exist at all for that question.

Proposed Response

Design change	Expected benefit	Trade-off introduced
Record <i>when</i> each memory was saved, and give newer facts priority over older, contradicting ones	Resolves the changed-preference and contradiction failures correctly, without relying on lucky wording	Adds storage/retrieval complexity; needs a clear rule for what counts as "replacing" an old fact versus being a separate, unrelated fact
Filter for sensitive information <i>before</i> saving anything, not after	Prevents passwords and similar secrets from ever entering storage	Requires a classification step at write time; risk of over-filtering legitimate information; adds latency when saving
Replace the blind cutoff with budgeting that prioritizes by relevance/importance, not just "first to fit"	Important facts survive a tight budget instead of being crowded out by irrelevant decoys	More complex ranking logic; some added latency when selecting context

Falsification

These proposed changes would be shown unnecessary, or wrong, if:

- Testing across many more varied, realistically-worded contradictions shows that plain keyword-overlap ranking (with no timestamp at all) reliably selects the current fact anyway — suggesting explicit time-tracking adds complexity without real benefit.
- A sensitive-data filter produces so many false positives in practice (blocking harmless facts) that it does more harm to usefulness than the leakage it prevents — suggesting a different approach (e.g. warning the user at save time, rather than blocking) is better.

Summary

2 of 10 cases are confirmed, clean failures (6 and 9). 1 case is a fragile pass that only worked because the underlying model is capable, not because the system is sound (2). 2 cases were initially inconclusive due to a flaw in our own test wording, and once corrected, one of them (3) also became a confirmed failure. 5 cases passed cleanly and are worth keeping as evidence that naive

keyword search is not broken across the board — only in the specific ways predicted by Deliverable 1's first-principles analysis.

Every improvement proposed above traces directly back to one of these three observed failures, matching the handbook's completion standard for this deliverable.